

Programming Project for COMP4621 2025 Spring

Lunxi CAO
lcaoar@connect.ust.hk

We aim to build a system that allows both chat and file sharing services. For file sharing, each client queries the file information from the server by UDP and gets the file by peer-to-peer TCP connection with another client. For chat, each client connects to the server by TCP. The server takes responsibility for forwarding received message by TCP to other users or the specified user.

1 Requirements

1. Programming must be in C.
2. You must implement rdt3.0 based on UDP to deal with commands about services and file sharing. For chat, the three commands should be implemented with TCP.
3. The server should be able to provide services to multiple clients simultaneously. That means it should provide a concurrent rdt3.0 based on UDP for file sharing, and concurrency on TCP using poll() for chat.
4. You must implement a p2p server and p2p client in “client.c”. Start the p2p server using the multi-thread function when the client get started.
5. The p2p server and the chat server must be based on poll() to enable concurrency.
6. The submission contains “client.c”, “server.c”, and a report “<your student id>.pdf”.
7. Your code is only required to process valid commands.

2 Grading Scheme

1. The three chat commands work well. We will manually test your code with valid commands in the demo storyline, including REGISTER, UPDATE, @ALL:, WHO, and @. The other file sharing commands are excluded. So, you can try yourself following the demo storyline. (12% for @ALL:, 8% for WHO, and 12% for @)

2. Your code passes the provided test1 for the simplest scenario. This test case will test your code with two clients without packet loss and without latency. Both these two clients connect to the server one by one, and one client downloads a file from another client. (6%)
3. Your code passes the provided test2 for concurrency of the server. This test case is like the above one. Except that these two clients connect to the server at the same time. (6%)
4. Your code passes the provided test3 for multi-threading of p2p client and server. There are three clients start one by one. Client2 downloads a file from client1 and client3 downloads a file from client2 at the same time. And they are required to finish the transfer within 10s. (6% + another similar case for 6%)
5. Your code passes the provided test4 for packet losses. We will test your server code with a modified client, which will drop some ACKs for REGISTER messages. (6% + another similar case for 6%)
6. Your code passes the provided test5 for packet losses. We will test your client code with a modified server, which will drop some ACKs for REGISTER/UPDATE/QUERY messages. (6% + another similar case for 6%)
7. A report for this programming project. (20%)